Data Mining and Knowledge Discovery (2026) 40:13
https://doi.org/10.1007/s10618-025-01173-9

Bijective graph learning architecture with multi-level attributes interaction

Ikenna Oluigbo[1] · Stefan Duffner[1] · Kajal Eybpoosh[1] · Catherine Pothier[1]

Received: 26 March 2025 / Accepted: 1 November 2025

Abstract
Techniques for representing graphs and preserving different graph features in low-dimensional embeddings have garnered much popularity and application over the years. The resulting embeddings can be applied to a wide variety of downstream machine learning tasks such as node classification and community detection. Despite the successes of Graph Neural Networks (GNN) in graph learning, this classical architecture is plagued with limitations including but not limited to issues of oversmoothing from message passing at increasing layer depth, dependence on scarce labeled data, long-range dependencies, and a large execution time resulting from long graph updates. By way of an alternative solution to the highlighted limitations, we propose an improved graph representation technique using a mapping bijection function with log smoothing to aggregate neighborhood properties and update node states in a message passing process, akin to that of a classical GNN. Similarly, to instantiate each node's class value, we adopt the unsupervised affinity propagation algorithm; with these class values an important component of the bijection function. The resulting state for each node at every update level is a simple encoded integer that captures in itself the properties of the neighbor nodes. Because the final node states are high-dimensional, they are transformed into low-latent vector representation, while preserving rich and meaningful features of the input graph. Through experiments with real-life datasets, we show that our proposed model outperforms existing models on node classification and community detection tasks.

Keywords Bijection function · Node classification · Graph learning · Attributes embedding · Community detection

Extended author information available on the last page of the article

Published online: 12 January 2026

1 Introduction

Graphs (or Networks) are mathematical structures which represent the interactions between different entities. Due to the adaptability of graphs, they can serve as a framework for representing and understanding relational entities in a variety of use cases from multiple domains such as protein-protein interaction networks, social networks, communication networks, Internet of Things, financial domain etc. These networks, when modeled as graphs, are non-linear data structures made up of nodes and edges, having high-dimensional properties. For instance, a road network can have cities as the nodes with each city having attributes such as city name (textual), year (integer number) etc., and the connection between cities as edges having property such as distance (float number). Adjacency matrix and adjacency list were the traditional methods of representing the high dimensional attributes of graphs in memory (Oluigbo et al. 2019), however, these methods preserve attributes in a high-dimensional latent space, resulting in high computational processing complexity and difficulties in extracting useful patterns from the preserved data. Also, it is very challenging to apply machine learning algorithms directly to graph structures for downstream tasks because machine learning algorithms work more effectively with low-dimensional data (Enrico and Alice 2024).

Owing to the peculiarity of graph structures and the challenges involved in preserving and extracting useful information from high dimensional entities, many researchers have tried to propose different methods to learn and preserve the high dimensional graph data in a low latent embedding, suitable for application to specific downstream machine learning tasks such as classification, prediction, community detection, recommender systems etc. Early studies on dimensionality reduction techniques such as Principal component analysis (PCA), Multi-dimensional scaling (MDS) and Local linear embedding (LLE) propose approaches that aggregate high-dimensional local neighborhood properties to try to learn global information and maintain pairwise distances between neighborhood entities(Tenenbaum et al. 2000; Yanning et al. 2018). Other approaches use natural language processing techniques (i.e. Skipgram) to learn low latent representations of an input graph. DeepWalk (Perozzi et al. 2014) and Node2Vec (Grover and Leskovec 2016) use uniformly truncated random walk and 2nd-order random walk respectively to explore neighborhood depths to build a node corpus (sequence of nodes), from which δ-dimensional embeddings for nodes are generated using Skipgram (Mikolov et al. 2013). Deep learning techniques such as SDNE (Wang et al. 2016) and DHNE (Tu et al. 2017) use traditional autoencoders which take a feature array to generate a low δ-dimensional latent representation of a graph granularity of interest, while minimizing the reconstruction error between the input data and learned output. The Variational Graph Autoencoder (VGAE) proposed in Thomas and Max (2016) is an unsupervised learning paradigm leveraging on the classic variational autoencoder for encoding and decoding vectors or images. The authors in Hui et al. (2023) use a variational graph autoencoder model with Laplacian filter feature smoothing to embed nodes and attributes in low-dimensional space.

Graph Neural Networks (GNN) and subsequent different variants have gained massive acceptance in the past few years. GNNs rely on *"message passing"*, a technique that involves exchanging information via local neighboring nodes or edges

to influence each other's updated embeddings, to preserve graph properties in low-dimensional space. There are different variants of GNN proposed in the literature. For instance, the authors in Jie et al. (2018) propose *FastGCN* (leveraging Graph Convolution Network), to derive an embedding functions for graph granularities, through performing Monte Carlo approximations on the graph integral transforms. Similarly, the authors in Petar et al. (2017) propose *GAT*, a self-attention architecture which generates latent representations for each node by attending over its neighbors. The resulting representation is applied to a node classification task. Inspired by the GNN architecture, we propose an approach which aims at tackling some of the limitations of classical GNN. The idea behind our proposed graph learning via attribute mapping technique, which we have given the acronym **GLAB**, is to generate representations for each node v in the network by mapping to itself the multiset of attributes which are direct neighbors of v. The aggregation function is the generalized cantor bijection function, with a log smoothing to reduce computational complexity (Ikenna et al. 2022). Specifically, GLAB's initial phase involves interpreting attributes for each node as a class value for suitability to the aggregation function. With each iterative graph update, the model enriches the encoded representation for each node in the graph using the aggregated properties of its neighbors $N(v)$. For γ number of iterative updates, the representation for node v would have been enriched γ number of times with the local and global properties of the network. The model uses a single feed-forward embedding layer as an end component, to generate the final embeddings for each node from the enriched encoded representation. The embedding layer ensures that the embeddings can be generalized across different machine learning tasks. Some of the interesting properties of GLAB includes: (1) No dependence on labeled data; (2) Low computational complexity due to representation smoothing; (3) The model can be applicable to nodes irrespective of their structural positioning in the network; (4) The model is permutation invariant to graphs; (5) The model is applicable to connected or unconnected graphs; and (6) The inferred embeddings can be directly applied to downstream tasks. Our contributions are as follows:

1. We propose an efficient and computationally inexpensive graph representation model using an aggregator bijection function with log smoothing, to learn low-dimensional latent embeddings of nodes in a network, while also preserving the attributive properties in the embedding.
2. We evaluate the performance of our model with state-of-the-art methods using real-life datasets, on selected downstream tasks.

The rest of the paper is organized as follow: Sect. 2 discusses related literature. Section 3 introduces our proposed model. In Sect. 4, we present the datasets used, the results of our evaluation and a discussion of the results. We conclude the paper in Sect. 5, and highlight possible directions for future work.

2 Related literature

Graph representation learning: There are a plethora of approaches for embedding nodes and subgraphs, while preserving the attribute property of these graph elements in the learned embedding; that is nodes or subgraphs with similar attributes will have similar embeddings and lie closely on the latent space. In Bijaya et al. (2018), the authors proposed a technique for preserving the neighborhood and structural property of subgraps in the embedding space. The technique uses truncated random walks to build the node corpus. While their experiments show relative improvement in performance over the state of the art, it can be argued that random walks have some limitations such as sampling too many nodes repeatedly (especially for disconnected networks) which can lead to a large variance for the algorithm, and also a truncated random walk is unable to explore the large depth in the graph to aggregate global properties. The authors in Perozzi et al. (2014); Grover and Leskovec (2016) proposed a node embedding technique that preserves the structural equivalence and homiphily property of the network. Aside the obvious challenges of truncated random walk which was used in both studies, the works failed to take cognizance of the attributes of nodes in the graph. The authors in Ikenna et al. (2023) describe a flexible approach for preserving the structural identities of nodes in a network on the embedding space, with nodes having similar structural identity mapped closely. The approach adopts the Poisson distribution measure with divergence estimation score to capture nodes structural similarities at k-hop proximity. While structural identity represents the characteristic behavior of nodes, it does not take into account the attributive property of the nodes. A non-negative matrix factorization embedding technique proposed in Sambaran et al. (2018) preserves the structure and semantics on the network in the learned embedding. Matrix factorization frameworks are generally computationally expensive for massive networks, and they can be sensitive to noise in the data which invariably decreases their accuracy.

Neighborhood feature aggregation:In the literature, there are several representation learning approaches that incrementally aggregate neighborhood information as part of their learning process. Most prominently, classical Graph Neural Networks (GNN) use the message passing technique to pool information within the GNN layer, allowing neighboring nodes to exchange information and update each other's embeddings. Basically, the message passing technique gathers all the neighboring node embeddings, aggregate the embeddings via an aggregation function (sum, mean, max, attention, or convolution), and then pool the embedding through an update function. The "sum" aggregation function adds the feature vectors of neighboring nodes but this may lead to unstable node embeddings when scaling up the graph features larger than those seen during training. The "max" aggregation function takes the maximum value for each feature vector across the neighbors thus emphasizing on the most important features but the function does not consider the whole neighborhood relevance to the feature vectors. The "mean" aggregation function computes the mean of the feature vectors of neighboring nodes thus failing to preserve the unique property of each neighbor nodes. An attention function calculates a weighted mean of the neighbors features based on how important each neighbor node ($N(v)$) is to the source node (v). And finally, the convolution function uses learnable filters to capture

important patterns and features within the neighborhood feature vectors (Dennis and Floris 2024; Isabelle and Matthias 2019; Wei-Lin et al. 2019). In addition, training a GNN to capture the expressiveness and patterns from the adjacency matrix of a large graph, which can be computationally complex and cumbersome.

We propose a technique which is efficient, flexible and adaptable to connected or unconnected networks. Without the need for a biased aggregate function and some of its drawbacks, we introduce a bijection function which directly maps the embedding from successive neighboring nodes ($N(v)$) onto a source node (v) in one pass through the network, thus ensuring that the global information of the network is considered when updating the embedding for a node in a subsequent pass through the network. An illustration of the approach is shown in Fig. 1. It shows the exchange of information within a neighborhood of nodes. Figure 1a depicts a sample network with randomly assigned attributes value. This attribute value is the initial node state. In Fig. 1b, every node updates its state by pooling information from its neighbors. This is done in parallel through the network in one network pass, thus reducing the computational complexity of updating node states in sequence of n-hop neighborhood depth as seen with many variants of GNN. Through this parallel update, a node would invariably improve its state with the global information of the network because the node's neighbors would have updated their own states with information from their neighbors as shown in Fig. 1b, thereby preserving important property of the network. Figure 1c describes the neighborhood information to node mapping, from which the final embedding is derived for each node as shown in Fig. 1d.

3 Problem definition and notations

In this work, we shall use the terms Graph and Network, as well as node and vertex interchangeably to mean the same thing. In this section, we define some relevant concepts, and thereafter we summarize the notations used in this study.

Definition 1 An **attributed graph** is a 3-tuple $G = (V, E, \mathbb{Z})$, where V consists of the set of vertices in the graph such that $V = v_1, v_2, \ldots, v_n$, E consists of the set of edges connecting the vertices such that $(u, v) \in E$ and $E \subseteq V \times V$, and $\mathbb{Z}$ is the set of attributes associated with each node. The attributes describe the auxiliary informa-

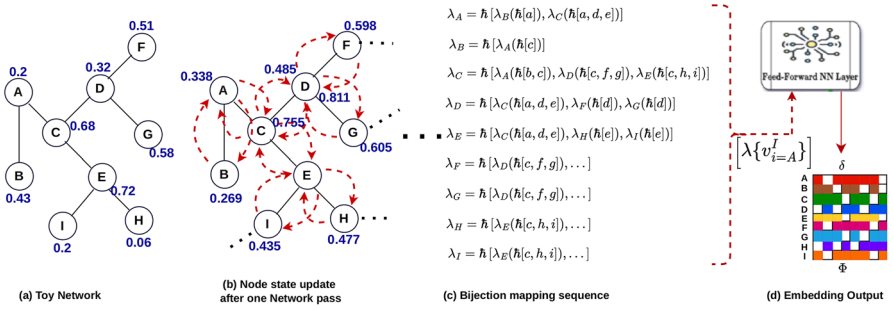


Fig. 1 Illustration of our proposed Network Pass between neighboring nodes

tion contained in nodes, aside the structural relations of node pairs. Formally, $\mathbb{Z}_{(u)}$ represents the attribute of vertex *u* in a graph.

Definition 2 An **undirected graph** is a graph with bidirectional edges. Given two nodes *u* and *v* in *G*, the edges (*u*, *v*) and (*v*, *u*) are equivalent. On the contrary, a **directed graph** has edges that are not equivalent. Our approach can cope with directed and undirected graphs.

Definition 3 Given a graph $G = (V, E)$, the **First-order proximity** defines the local relationship between directly connected vertices in the graph. The probability distribution of first-order proximity for two vertices *u* and *v* is defined as:

$$Pr(u, v) = \frac{1}{1 + \exp(X_u^T \cdot X_v)} \tag{1}$$

Definition 4 Given a network $G = (V, E)$, a **subgraph** $G_i = (V_i, E_i)$ can be defined as a subset of *G* , denoted as $G_i \subseteq G$, if and only if $V_i \subseteq V$ and $E_i \subseteq E$.

Definition 5 **Class values** (or **labels**) for nodes in *G* are integers, alphanumeric, or continuous variables which are assigned to nodes $\{v_1, v_2, v_3, ..., v_n\} \in |V|$, to properly distinguish a node from other nodes in *G*. These class values are characteristic attributes of nodes.

The node attributes can be of any type and represent the information about the nodes in the network. Nodes are often in neighborhoods with other nodes. The first-order neighbors of a node *v* denoted as *N*(*v*) are all the nodes directly connected to *u*. The toy network in Fig. 1a represents an undirected network with nodes having attribute information where V = {A,B,C,D,E,F,G,H,I}, E = {(A,B), (A,C), (C,D), (C,E), (D,F), (D,G), (E,H), (E,I)}, and $\mathbb{Z}$ = {0.2, 0.43, 0.68, 0.32, 0.72, 0.51, 0.58, 0.06, 0.2}. Table 1 summarizes the principal notation of this paper.

4 GLAB framework

In this section, we present our proposed technique for learning the global attributes of nodes through a mapping bijection function. We describe the learnable parameters, the mapping and information passing process shown in Fig. 1, and how the final node states are embedded, followed by the computational complexity of our model.

4.1 Overview of framework

Figure 2 illustrates the general idea of the GLAB model. The first step of the model is an initial random prediction of each node's class values (ℓ) from their attributes. This process is done in an unsupervised learning manner. The initial random class values are important for the bijection mapping function (λ). Next, we build a neighborhood

Table 1 Notations used in this work

Notations	Descriptions
G	Attribute Graph
V	Set of Nodes in G
E	Set of Edges in G
n	Number of nodes
$\mathbb{D}$	Gaussian distribution
$\mathbb{Z}$	Set of Attributes of Nodes in G
γ	Used to denote the number of iterations through G
$\ell/\hat{\ell}$	Generated class values for Nodes in G
$N(v)$	Neighbor of node v
$N(\ell)$	Adjacency list of Neighbor nodes class values
δ	Latent Embedding dimension
Γ	Function to generate new class values
$\hat{h}_v$	Bijection function for node v
λ	Neighborhood information to node mapping function
Φ	Learned attribute embedding

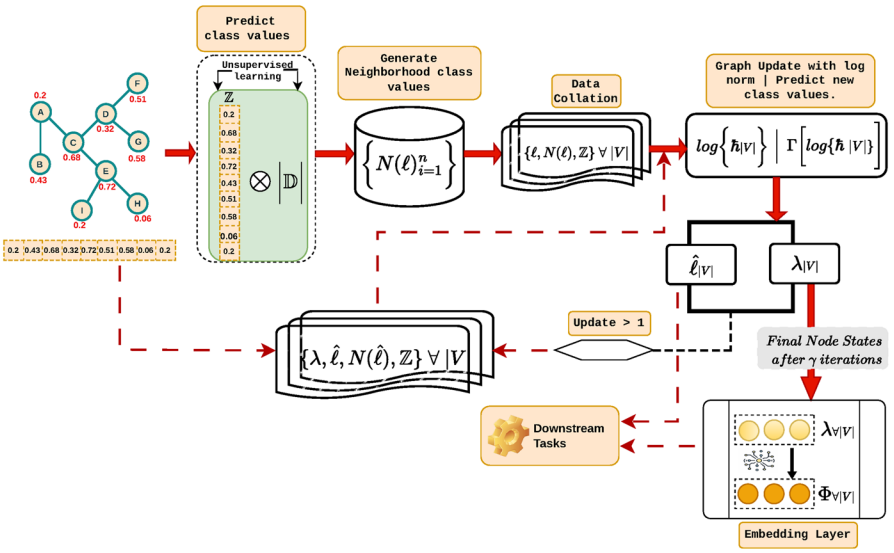


Fig. 2 The main framework of GLAB

class value corpus represented as an adjacency list. The initial class values of the nodes, the nodes' attributes, and the neighborhood class values are collated for the graph update task. The graph update computes the node states for each node from the neighborhood information using the bijection function, with a log smoothing to normalize the node states. The result of this process is a corpus of node states and refined class labels, which is an improvement over the initial randomized class value. To further improve the node states for each node, depending on the size of the graph, further graph updates could be performed with more robust information comprising the current states of all nodes, refined class values of nodes, node neighborhood class

values and the node attributes from the input network. Upon completion of the graph updates, the final node states are passed through a feed-forward embedding layer to generate δ-dimensional representations of each node v. Ultimately, these embeddings and the generated labels can be used for relevant downstream machine learning tasks.

Parameters descriptions:

The input graph to our model is an attributed graph, with each node having characteristic properties known as **attributes**. These attributes describe the role a node plays in the network. We aim at preserving this property in the final embedding such that nodes performing similar roles in the network will have similar embeddings. We can furthermore choose to apply an unsupervised technique to group similar nodes into a community. An additional output from the model are the node labels learned in an unsupervised manner. Generally, nodes in the same community will have the same label. In this study, the input node attributes take the form of an integer or real number. To properly represent the node attributes as continuous quantities suitable for initial class value instantiation, we perform a dot product of the attributes and a discrete Gaussian distribution denoted as $[\mathbb{Z}_{i=1}^{n} \cdot \vec{\mathbb{D}}]$.

4.2 Class value instantiation

In this section, we discuss the technique for generating class values from the input attributes. The process of mapping node neighborhood information to a target node rely on the bijection function (to be discussed in subsequent sections), which uses the unique class value of each node. To generate the class values of the nodes, we adopted an unsupervised Affinity Propagation Algorithm. This unsupervised clustering technique examines intricate, non-linear distribution patterns in data and uses a "message passing" procedure between data points to identify cluster centers (or exemplars) and assign data with similar property around the exemplar to form a cluster community (Frey and Dueck 2007). We favor the Affinity Propagation technique for several reasons: (1) it automatically identifies cluster centers and cluster communities, with the community value inherited by all nodes in that community, (2) it can relatively well handle datasets with noise and outliers, (3) it uses a similarity matrix to find similarity in intra-community nodes (4) the similarity matrix can either be symmetrical or asymmetrical (Hong and Zhang 2016). Instantiating class values through Affinity Propagation takes place in three steps:

1. The first step involves computing a similarity matrix, with each entry corresponding to the similarity score (s) between two data points based on their features. The similarity score is the negative of the square of distances between the data points. Given two nodes i and j, the similarity score is calculated as

$$s(i, j) = -\|(\mathbb{Z}_i \cdot \vec{\mathbb{D}}) - (\mathbb{Z}_j \cdot \vec{\mathbb{D}})\|^2 \tag{2}$$

   where $\mathbb{Z}_i$ and $\mathbb{Z}_j$ are the attribute set of node i and node j respectively.
2. From the resulting similarity matrix, the suitability of one node to be a cluster center (or exemplar) is estimated. The result is a responsibility matrix that is

updated throughout the iteration process of the affinity propagation algorithm. To determine if node *i* is a suitable exemplar for node *j*, the update process is defined as

$$r(i,j) \leftarrow s(i,j) - max\left[a(\mathbb{Z}_i \cdot \vec{\mathbb{D}},\ \mathbb{Z}_{j'} \cdot \vec{\mathbb{D}}) + s(\mathbb{Z}_i \cdot \vec{\mathbb{D}},\ \mathbb{Z}_{j'} \cdot \vec{\mathbb{D}})\ \forall\ j' \neq j\right] \quad (3)$$

here, $a(\mathbb{Z}_i \cdot \vec{\mathbb{D}},\ \mathbb{Z}_{j'} \cdot \vec{\mathbb{D}})$ is the availability matrix which is initialized to 0.

3. Having derived the responsibility matrix for the set of data points, an availability matrix, initially set to 0 can be computed to determine the most suitable exemplars for each derived cluster. This matrix is computed as

$$\begin{aligned} a(i,j) &\leftarrow min\left[0, r(\mathbb{Z}_j \cdot \vec{\mathbb{D}},\ \mathbb{Z}_j \cdot \vec{\mathbb{D}}) + \sum_{i' \notin i,j} max\{0, r(\mathbb{Z}_{i'} \cdot \vec{\mathbb{D}},\ \mathbb{Z}_j \cdot \vec{\mathbb{D}})\}\right]\ \forall\ i \neq j\ \ and \\ a(j,j) &\leftarrow \sum_{i' \neq j} max\{0, r(\mathbb{Z}_{i'} \cdot \vec{\mathbb{D}},\ \mathbb{Z}_j \cdot \vec{\mathbb{D}})\} \end{aligned} \quad (4)$$

here, $r(\mathbb{Z}_j \cdot \vec{\mathbb{D}},\ \mathbb{Z}_j \cdot \vec{\mathbb{D}})$ is the responsibility of node *j* being an exemplar to itself, while $max\{0, r(\mathbb{Z}_{i'} \cdot \vec{\mathbb{D}},\ \mathbb{Z}_j \cdot \vec{\mathbb{D}})\}$ is the responsibility of other data points i' to the potential exemplar *j*.

The responsibility and availability updates are done iteratively until cluster limits remain unchanged over a number of iterations, or until a predefined number of iterations is reached. Lastly, a criterion matrix computed as the aggregate of both the responsibility and availability matrix is used to determine the final clusters and the centroid for each cluster. As mentioned above, nodes belonging to the same cluster have the same class value. The criterion matrix is represented as

$$c(i,j) \leftarrow r(i,j) + a(i,j) \quad (5)$$

4.3 Neighborhood aggregation and normalization

Neighborhood Aggregation is an integral part of our proposed model. The semantic neighborhood features of a node is an important parameter for updating the node's state; we therefore need a technique capable of encoding these semantic features in a form suitable for such update. The concept of neighborhood aggregation involves encoding the neighborhood information of a vertex into a simple numerical integer. This encoding is used to update the internal state of each node in every graph update process. There are three advantages over classical embedding techniques: (1) we eliminate the need of grid hyper-parameter estimation of the dimensionality embedding space which can result in poor model performance, (2) we circumvent the need for complex computationally expensive encoding techniques, and (3) Since only one-hop neighbors are needed for updating a node state, we necessarily do not have to load all the graph into memory. When a new node is added to the neighborhood, it

is easy to isolate only that neighborhood and encode the neighborhood information. This further reduces the computational complexity of the overall model.

To encode the neighborhood information for updating a node state, we adopt the generalized cantor polynomial, which in our study we defined as

$P_k : \{\ell \| N(\ell)\}^k \longrightarrow \mathbb{N}$

$\forall (v_1, v_2, v_3, \cdots, v_k) \in \{\ell \| N(\ell)\}^k$ and $k > 0$

$$P_k(v_1, v_2, v_3, \cdots, v_k) = \sum_{j=1}^{k} \hbar(j, v_1 + v_2 + v_3 + ... + v_j) \tag{6}$$

where

$$\hbar(s, t) = \begin{pmatrix} t + s - 1 \\ s \end{pmatrix} = \frac{(t + s - 1)!}{s!(t - 1)!} \tag{7}$$

Equation 6 is a bijection function. To demonstrate the application of Equation 6 in encoding neighborhood information of a node, we define the following concept:

Definition 6 The **one-hop neighborhood** of a vertex v in G is the multiset of class attributes appearing in the direct neighbors of v in G. Two vertices u and v have the same one-hop neighborhood in G if they have exactly the same multiset of class attributes on their neighbors.

One-hop neighborhood is defined as a multiset of class attributes because more than one neighbor of a node can have the same class attribute (Ikenna et al. 2022). Therefore, *u* and *v* in *G* have the same encoded λ, i.e. $\lambda(u) = \lambda(v)$ if and only if *u* and *v* have similar one-hop neighborhood. Figure 3 show the concept of one-hop neighborhood for vertices u_1 and u_2 having similar class attributes $\{ABCB\}$ from neighboring vertices. Figure 4 depicts a sample encoding of 7 vertices (*a*, *b*, *c*, *d*, *e*, *f*, *g*), with each vertex having class value 1, 2 or 3. These class values corresponds to the instantiated value for each node as described in Sect. 4.2. From Eq. 6, *k* denotes the number of distinct class values i.e. $k = 3$. From Fig. 4, *j* in $\hbar(j, v_1 + v_2 + v_3 + ... + v_j)$ corresponds to a vertex class value, and v_j is the number of apparition of attribute *j* in

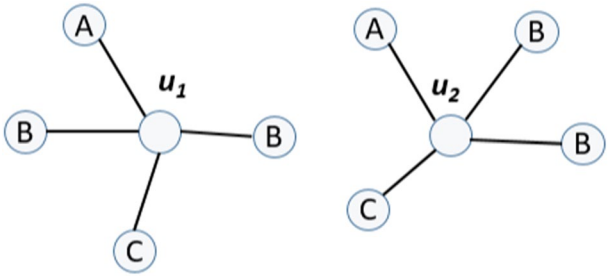


Fig. 3 Vertices u_1 and u_2 having similar One-hop neighborhood. Image adapted from Ikenna et al. (2022)

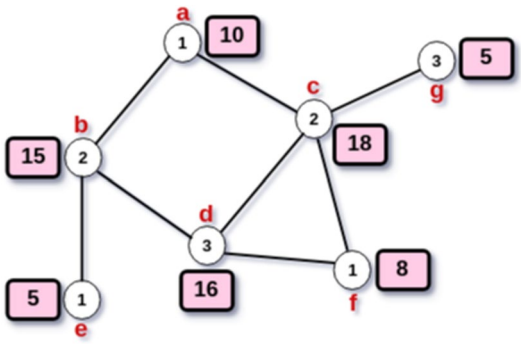


Fig. 4 Encoded Neighborhood Information for nodes a–g

the direct neighborhood of vertex *v*. We compute the encoding for vertex *a* from its neighborhood information as follows:

$\lambda(a) = P_3(0, 2, 0,) = \hbar(1, 0) + \hbar(2, 0 + 2) + \hbar(3, 0 + 2 + 0) = 0 + 4 + 6 = 10$
We see that class values 1 and 3 are not in the neighborhood of node *a* while class value 2 appears 2 times. To encode the neighborhood information for nodes *b* to *g*, we follow the same technique:
$\lambda(b) = P_3(2, 0, 1) = \hbar(1, 2) + \hbar(2, 2 + 0) + \hbar(3, 2 + 0 + 1) = 15.$
$\lambda(c) = P_3(2, 0, 2) = \hbar(1, 2) + \hbar(2, 2 + 0) + \hbar(3, 2 + 0 + 2) = 18.$
$\lambda(d) = P_3(1, 2, 0) = \hbar(1, 1) + \hbar(2, 1 + 2) + \hbar(3, 1 + 2 + 0) = 16.$
$\lambda(e) = P_3(0, 1, 0) = \hbar(1, 0) + \hbar(2, 0 + 1) + \hbar(3, 0 + 1 + 0) = 5.$
$\lambda(f) = P_3(0, 1, 1) = \hbar(1, 0) + \hbar(2, 0 + 1) + \hbar(3, 0 + 1 + 1) = 8.$
$\lambda(g) = P_3(0, 1, 0) = \hbar(1, 0) + \hbar(2, 0 + 1) + \hbar(3, 0 + 1 + 0) = 5.$

We also observe that nodes *e* and *g* have the same encoding since they have similar one-hop neighborhood of class values.

4.3.1 Normalization

In Sect. 4.3, we showed how the properties of a node's neighbor can be encoded into a simple numeric integer. Taking Fig. 4 as an example, after the first graph update, the result is an encoding with high integer values. Given that we might want to perform a few more graph iterations, we quickly see that our pairing function for each node will result into very large encoded integer features, resulting from the fact that the function computes factorials, as seen in Eq. 7. These large features can further increase the complexity of our model.

To overcome this problem, we introduce a logarithm transformation normalization, which basically scales the integer features into much smaller features. To normalize the features, we simply take the natural logarithm of the large integer value i.e. $\lambda(a)^{'} = \ln(\lambda(a))$. Normalization improves model performance, enhances the interpretability of the features, and reduces skewness in the node states, while preserving the important properties encoded in the features (Ikenna et al. 2022).

4.4 Information passing and mapping design

Many downstream machine learning tasks such as classification and community detection require that the learned embedding is aware of the neighborhood features and connectivity of the graph. This can be achieved through an information passing and mapping process, in which neighboring nodes exchange information and significantly influence each other's final learned embeddings (Gilmer et al. 2017).

The information passing and mapping technique for a graph G can be described in a series of steps:

1. The process starts with the initial class values computed for all nodes $\{\ell_1, \ell_2, \ell_3, ..., \ell_n\}$ in G, as elaborated in Sect. 4.2.
2. For each node, collect its class value, attributes, and class values of the neighbors. Therefore, for node $v \in |V| : \|\ell_v, \mathbb{Z}_v, N(\ell_v)\|$. We assume that the initial node state s_0 is the attribute of the node, i.e. $s_0 = \mathbb{Z}_1, \mathbb{Z}_2, \mathbb{Z}_3, ..., \mathbb{Z}_n \ \forall \ |V| \in G$.
3. The collated information for all nodes is passed through an update function. The update function consists of the following:

   - a bijection function encodes the one-hop neighborhood information of each node into an integer from Eqs. 6 and 7.
   - a log transformation function which normalizes the encoded numerical feature to smaller scale. The normalized value is the new state s_1 for each node. The new node state is described as $s_1 = \ln \lambda(v_{i=1}^n) \ \forall \ |V| \in G$.
   - using the normalized encoding, the model predicts a new class label for the nodes $\{\hat{\ell_1}, \hat{\ell_2}, \hat{\ell_3}, ..., \hat{\ell_n}\}$ in graph G. The intuition is that we are able to predict more accurate node labels after iterating through the graph, since each node state has been updated with its rich one-hop neighbor information; with each neighbor node also having updated its state with its own one-hop neighbors.

   - In G, a node v will therefore have the following outcomes:

$$
\begin{aligned}
s_{0_v} &= \mathbb{Z}_v \\
s_{1_v} &= \ln \lambda(v) = \ln \left(\sum_{j=1}^{k} \hbar(j, v_1 + v_2 + v_3 + ... + v_j) \right) \\
\ell_v &= r(s_{0_v}, \mathbb{Z}_{v=1}^n \in G) + a(s_{0_v}, \mathbb{Z}_{v=1}^n \in G)
\end{aligned}
$$

   The same process follows for all nodes in G.
4. To enrich the quality of the node embeddings, a further update to the node states may be required. If each node is able to update its state with the global information of the graph through its one-hop neighbors, our model is able to optimally preserve and embed in the embeddings the similarity properties of nodes. Hence, nodes with shared characteristic attributes will have similar embedding. To perform the next node state update, the outputs from the immediate last update are fed back into the model. As shown in Fig. 2, to perform a next update for node v, the new node state s_{1_v} becomes the input attribute to the affinity propagation

algorithm. However, we also feed the original attribute $\mathbb{Z}_v$ into the algorithm. This important feedback loop enable the model retain some information about the original network while computing the next node states and predicting refined class values. Therefore, adopting Eq. 5 to predict an improved class value $\hat{\ell}_v$, the criterion matrix given two node *u* and *v* at second graph update is defined as:

$$\begin{aligned} c(u,v) \leftarrow r(\|\mathbb{Z}(u) \cdot \ln \lambda(u) \otimes \mathbb{D}\|, \|\mathbb{Z}(v) \cdot \ln \lambda(v) \otimes \mathbb{D}\|) \\ + a(\|\mathbb{Z}(u) \cdot \ln \lambda(u) \otimes \mathbb{D}\|, \|\mathbb{Z}(v) \cdot \ln \lambda(v) \otimes \mathbb{D}\|) \end{aligned} \tag{8}$$

For all nodes in *G*, the $\hat{\ell}$, the neighbor class values for each node $N(\hat{\ell})$, and the node states $\ln \lambda$ are collated, and passed through the processes in step 3 to compute new node states s_2 for all nodes, and even more refined node labels $\hat{\ell}_2$. After γ number of graph iterations, the final node labels $\{\hat{\ell}_1, \hat{\ell}_2, ..., \hat{\ell}_n\}$ can be used for supervised downstream tasks, while we pass the final node states $\{\ln \lambda_1, \ln \lambda_2, ..., \ln \lambda_n\}$ through a feed forward embedding layer to transform the sparse, high-dimensional node states into low-dimensional vector embeddings of fixed length. The meaningful semantic properties from the input node states is preserved in the output learned embeddings (Φ), and these δ-dimensional embeddings are used in related downstream tasks.

4.5 Learning representations from node states

The final part of the model involves producing embeddings for each node from their final computed node states, with the node properties preserved in the embedding. To achieve this, we pass the node states into the embedding layer of a feed-forward neural network. Using the embedding layer comes with a number of benefits, including (1) reducing data dimensionality, (2) flexibility as a standalone component of a neural network architecture (3) capturing and preserving semantic properties, and (4) through the preserved semantics, learned embeddings can generalize well to related downstream task.

More specifically, through the embedding layer, we are able to transform (with initially randomized weights) the non-negative node states into δ-dimensional continuous vector representations of the input states. These vector representations encode the semantics relationships and patterns from the input states. Given a node *v*, the vector embedding Φ_v for *v* can be computed through Eq. 9:

$$\Phi_v = \sigma(W \cdot \lambda_v + b)/\|\Phi_v\| \tag{9}$$

W and *b* are the initialized weight matrix and bias respectively, σ is the activation function, and $\|\Phi_v\|$ denotes the *l*2 normalization of the embedding.

To train the embedding layer, we use the following parameters.

1. The inputs of the embedding layer are the final node states of the nodes in *G*, the input dimension corresponding to *n* in *G*.
2. The output is a continuous δ-dimensional embedding, comprising *n* number of rows (one for each node), with each row having 32-dimensional length.

3. To ensure a non-linear transformation, the Rectified Linear Units (ReLU) activation function is used. With ReLU's output range being 0 to infinity, wide range of patterns in the input data can be captured.
4. The weights and bias are randomly initialized, with a *l*2 regularization.
5. We use the contrastive loss function to contrast the input node states, ensuring that similar node states have closely similar embeddings. The loss function is optimized with *Adam* optimizer. Generally, the contrastive loss function is defined as:

$$\mathbb{C}_{loss} = -\frac{1}{2L}\sum_{u=1}^{L}\log\frac{exp(sim(\Phi_i,\Phi_j)/\tau}{\sum_{v=1}^{2L}1_{u\neq v}exp(sim(\Phi_u,\Phi_v)/\tau} \tag{10}$$

where: Φ_i is the embedding of the ith node, Φ_j is the embedding of the j^{th} node, $sim(\Phi_i, \Phi_j)$ is a dot product similarity function, τ is a discriminative temperature parameter, and L is the batch size.

5 Experimental evaluation and result discussion

All experiments were carried out on Intel Core Ultra 5 135Hx18 processor machine, running Ubuntu 24.04.1 LTS. The machine has a 32GB RAM size and at a maximum clock speed of 5GHz. The experiments were carried out in a Python 3 environment and in the JuptyterLab IDE, while we also maintained the default parameter settings for all baseline models. We evaluate our model against *DeepWalk* (Perozzi et al. 2014), *Node2Vec* (Grover and Leskovec 2016), *GAE* (Thomas and Max 2016), and *PLANETOID* (Zhilin et al. 2016).

1. DeepWalk: This node embedding technique learns δ-dimensional representations through a first-order neighborhood random walk, to build a node corpus of fixed length. The corpus is then trained using the skipgram model (Mikolov et al. 2013) with hierarchical softmax activation function, and optimized using Stochastic Gradient Descent (SGD).
2. node2vec: This technique learns vector representation of nodes in low-dimensional space by simulating biased random walks guided by two hyperparameters ($InOut - q$ and $Return - p$ parameters) to collect node samples. The samples are trained using the Skipgram model with negative sampling, and optimized using Stochastic Gradient Descent.
3. GAE: This embedding technique uses the variational graph auto-encoder framework (GCN encoder and an inner product decoder) to learn low latent representation of graph-structured data for undirected graphs, in an unsupervised manner.
4. PLANETOID: This technique learn an embedding for each node in the graph, through the node's class label and its neighborhood properties. The technique offers both transductive and inductive learning.

Table 2 Network data statistics (Rossi and Ahmed 2015)

Dataset	Nodes	Edges	Dataset description
Citeseer	3264	4536	Scientific publications
Cora	2708	5429	Scientific publications
Proteins	43.5k	162.1k	BioInformatics
Enzymes	19.5k	75k	Protein structures

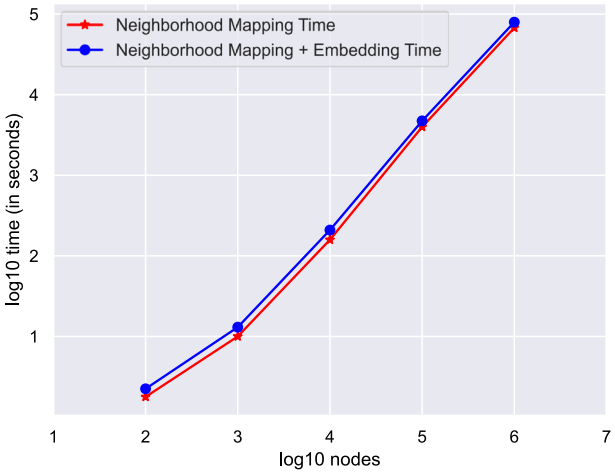


Fig. 5 Average execution time on Erdos-Renyi graphs

We evaluated all five models through the node classification and community detection tasks. The benchmark datasets used in this study are summarized in Table 2.

5.1 Runtime scalability analysis

Using the same parameters, we test the scalability of our model to different instances of the Erdos-Renyi graph. The Erdos-Renyi graph is a uniform graph with randomly instantiated group of nodes and its corresponding edges. In our experiment, each progressive instance of the Erdos-Renyi graph is an increasing number of nodes in the graph (i.e. from 100 to 1,000,000) by a factor of 10. To mimic an attributed network, we randomly assign attributes to the graph nodes. Embedding time is the additional time required to transform the node states into low-latent embedding. The execution time is displayed in a log-log scale. The result of Fig. 5 shows that our model scales very linearly, far better than the log-log constant $(1 + 1/t)^t$, where t is the number of nodes represented in the log-log scale. Our model not only has an optimal space complexity as node neighborhood can be processed separately without having to load the entire graph into memory; this result further shows the optimal time complexity of the model. Thus, our model can perform well when applied to massive attributed networks.

5.2 Multi-class node classification experiments

This section discusses the node classification results. As we have shown in Fig. 1a, the nodes in a network represent real-life objects, with each node having some peculiar characteristics. We perform node classification, a machine learning downstream task aimed at predicting distinct labels to nodes in a graph based on the characteristics of the nodes and the relationships between the nodes in the graph neighborhoods. To commence the classification task, we first learn vector embeddings for the nodes in our 4 baseline models, including vector embedding for our model. Next, we design a deep learning classification model to classify the nodes using the embeddings from all models. The deep learning model has the following parameters: The model has 3 hidden layers with a total of 28 neurons. We train the model with 100 epochs, with a sparse categorical cross-entropy and *Adam* optimizers. For all baseline models, we carry out the same experiment under similar environment and with the same parameters. Inputs to the classification model are the vector embeddings and ground-truth labels of the datasets. For quality and accuracy assurance, the classification experiment is repeated three times with 60 : 40, 70 : 30, and 80 : 20 train-test sets ratio. The classification result is evaluated using the Accuracy metric and F1 score. The accuracy metric measures the proportion of correctly classified nodes out of all the nodes in the graph, the F-1 score is the harmonic mean of the proportion of true positive predictions among all positive predictions (Precision) and the proportion of true positive predictions among all actual positive instances (Recall). F1 score is calculated as:

$$F1 = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

where

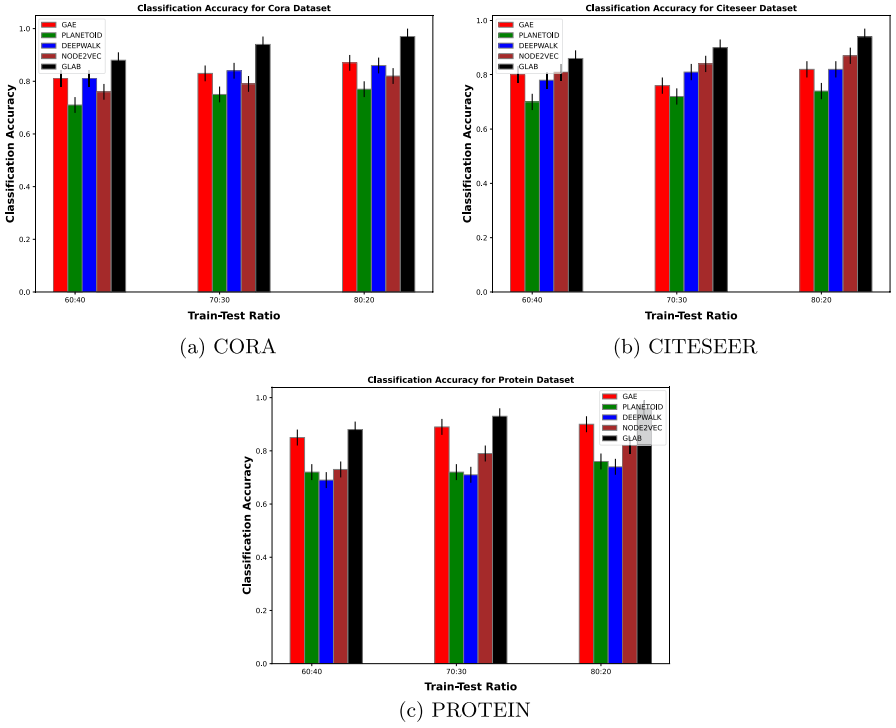
$$\text{Precision} = \frac{TP}{TP + FP};\ \text{Recall} = \frac{TP}{TP + FN}$$

$$(TP = \text{True Positive},\ FP = \text{False Positive}, FN = \text{False Negative})$$

The results of the multi-class node classification are shown in Table 3 and Fig. 6. We see that as we progressively increase the training dataset from 60 to 80%, our model outperforms other baseline models in the three datasets (Cora, Citeseer, and Protein) in predicting class labels for the nodes in the graph. Our model outperforms GAE by almost 10%; research has shown that GAE is not very scalable and cannot handle a very large network, as well as its inability to properly incorporate the node character-

Table 3 F1 score with enzymes dataset

Models	Precision	Recall	F1 score
GAE	0.82	0.86	0.84
PLANETOID	0.72	0.77	0.74
DEEPWALK	0.76	0.79	0.77
NODE2VEC	0.84	0.82	0.83
GLAB	0.91	0.95	0.93



(a) CORA (b) CITESEER

(c) PROTEIN

Fig. 6 Multi-class node classification accuracy metric results for Cora, Citeseer, and Protein datasets

istics in the learning algorithm (Thomas and Max 2016). On average, the DeepWalk and Planetoid models perform worse. DeepWalk uses an unguided random walk in sampling nodes in a corpus for embedding, and as such, it focuses solely on learning the structural patterns in the embedding while ignoring the attributes of the nodes. On the other hand, PLANETOID can suffer scalability limitations which affect the final embedding, while also being heavily sensitive to the time-consuming choice of parameters in the model, with embedding quality depending on these parameter settings. This is also true of Node2Vec which depends on tunable parameters, and lack of consideration for the node attributes of the graph. In addition, the results of the $F1$ score in the enzyme data set as seen in Table 3 show that our model has better positive predictions among all positive instances in the dataset, with a score of 93%. Our model is scalable to a large network as demonstrated in Fig. 6, no dependence on bias parameters, and effectively incorporates the different node attributes in the learning algorithm.

5.3 Community detection

We describe the community detection results in this section. This downstream task identifies nodes that are more densely interconnected and similar with each other (communities) than with other nodes in the network, and clusters the interconnected nodes together. While many community detection task are defined by the structural

relationship between nodes, we try to cluster nodes that share similar properties that are captured and preserved in the learned vector embedding. We designed a conventional centroid-based algorithm with a distributed stochastic neighbor embedding technique to map each data point of the input high δ-dimensional embedding into two dimensional space, which are then clustered in communities of similar nodes. The input of the community detection model is the δ-dimensional embedding from the baseline models. The result of Fig. 7 show the clustering of similar nodes into their distinct communities.

To evaluate the result of the community detection task, we use the Compactness Cohesion measure and the dissimilarity measure. The compactness cohesion measures how closely knit the objects within the same communities are. A small within-community variation generally indicates good compactness within the objects of the community. A visual examination of the community detection result of Fig. 7 shows that our model has a good measure of tight compactness for community objects, with little outliers compared to other baseline models. This indicates that nodes with similar properties are better grouped together in their separate communities. In contrast, the results of GAE and Node2Vec indicate variations and outliers within communities. A dissimilarity (or distance) matrix measure computes the pairwise distance between nodes in a community. A compact and dense community of similar nodes will have a small measure of dissimilarity compared to a loose community of nodes. More specifically, since the similarity properties of the nodes are preserved in the learned embedding, a minimal mean distance measure computed from the community centroid shows a compact community of nodes (Rebecca et al. 2015). Given two nodes u_1 and u_2 in the same community, the dissimilarity measure decreases if u_1 and u_2 are similar. To understand the dissimilarity index, the pairwise distance between

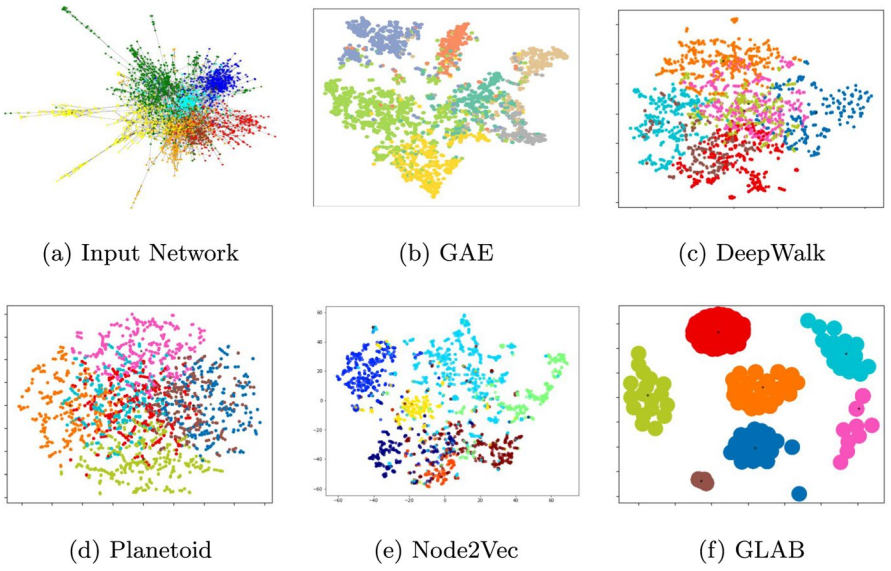


Fig. 7 Community detection from Cora network. **a** is the input network, and **b**–**f** show the communities detected by GAE, DeepWalk, Planetoid, Node2vec and our proposed method respectively

Table 4 Mean dissimilarity index with Cora Dataset

Model	Dissimilarity Matrix	Index
GAE	[3.319, 4.05, 3.830, 3.750, 4.174, 3.526, 3.875]	3.789
DeepWalk	[6.623, 6.870, 6.110, 6.848, 6.256, 6.118, 6.432]	6.465
Planetoid	[9.490, 9.159, 9.732, 8.988, 10.015, 8.802, 10.729]	9.559
Node2Vec	[4.439, 4.39, 4.480, 4.501, 4.477, 4.372, 4.355]	4.431
GLAB	[1.50, 1.385, 1.336, 1.539, 1.384, 1.315, 1.332]	**1.399**

Bold indicate statistical significance p-value < 0.05

nodes in each community is computed, showing the average distance between the nodes. The result of the dissimilarity measure is shown in Table 4.

The results in Table 4 clearly show that our model has the lowest average dissimilarity index score (p-value < 0.05), indicating a much compact community of nodes with minimal distance between them. The index score of the GAE and Node2Vec models closely follows our model. The Planetoid model shows the worst index score indicative of an average large distance within community nodes. This result is closely correlated with the community cohesion result in Fig. 7.

6 Conclusion and future perspectives

In this study, we propose a new technique for learning the attributes of a graph while preserving these rich and expressive properties in a learned embedding. To capture and preserve graph attributes, we introduce a bijection function to encode a node's neighborhood information into a normalized integer, which can then be used to improve the node states in γ number of graph iterations. We can then generate embeddings for each node using the corpus of node states. By iteratively interacting with the one-hop neighborhood of nodes, we can explore the global attributes of a graph, thereby capturing its rich attributes. To show the effectiveness of our model, we compare with other baseline models like GAE, PLANETOID, NODE2VEC, and DEEPWALK, in a multi-class node classification and community detection downstream tasks. In both experiments, our model achieves a better result compared to the other baseline models. This indicates that our model is capable of effectively capturing and preserving the global attributes of a network in the learned vector embedding.

Our proposed model is applicable in any network where the actors (or nodes) have some distinguishing characteristics, for which we intend to discover the intrinsic relationships between these characteristics in unsupervised means. For example, we might be interested in predicting influential connections or categories of items to which users might be interested in a social network; or to help users determine cities of interest from a collection of cities in a transport network; or perhaps to find targeted groups of emails sent within an organization. As demonstrated in our experiments, we learned representations of a biological protein network using the node (protein molecule) sequence information, from which we tried to understand the complex functional relationships of the nodes. Furthermore, our study is applicable

in the area of graph classification and visualization such that we can examine the visual representation of the graph to reveal patterns and outliers in the data for further investigation. This demonstration is shown in Fig. 7.

Some of the limitations of the proposed model include: (1) This study is limited to static attributed graphs, with unlabeled nodes. We focused on only node attributed graphs, where each node is associated with descriptive information, (2) We limited this study to preserving only the graph homophily properties in the learned embedding, without considering the structural setup of the network. This is due to the peculiarity of the techniques (bijection function) that we have adopted. (3) We have limited the node attributes used in this study to those in the form of numerical values (integers or continuous).

A future improvement on this study aims to provide solutions to these limitations. We want to expand our model to capture the global structure of the network as well as extend the model to temporal graphs to understand how graph attributes and structural properties evolve over short periods of time. We also want to incorporate edge information to further enrich the expressiveness of learned embedding. This learned embedding can be applied to graph level tasks. Finally, we aim at expanding the model to handle attributes in other formats, e.g. textual attributes. Through this, we would be contributing to the Natural Language Processing (NLP) research area by providing an alternative technique for NLP analysis.

Author contributions The study conception and design was performed by I.O. and S.D. Implementation and Data acquisition was performed by K.E. and C.P. Analysis was performed by I.O. and S.D. The first draft of the manuscript was written by I.O. and all authors commented on previous versions of the manuscript. All authors read and approved the final manuscript.

Funding This work has been funded by the ANR project REMATCH ANR-21-FAI2-0009 and the ANR project GRADIENT ANR-22-CE23-0009.

Data availability No datasets were generated or analysed during the current study.

Declarations

Conflict of interest The authors declare no competing interests.

Ethical approval All authors declare that they adhere to the ethical principles of the journal.

References

Bijaya A, Yao Z, Naren R, Aditya P (2018) Sub2vec: Feature learning for subgraphs. In: Pacific-Asia conference on knowledge discovery and data mining, pp 170–182

Dennis C, Floris B (2024) Effects of graph neural network aggregation functions on generalizability for solving abstract argumentation semantics. In: SAFA'24: 5th international workshop on systems and algorithms for formal argumentation. https://ceur-ws.org/Vol-3757/paper6.pdf

Enrico B, Alice G (2024) The challenges of machine learning: a critical review. Electron 13:1–30. https://doi.org/10.3390/electronics13020416

Frey BJ, Dueck D (2007) Clustering by passing messages between data points. Science 315:972–976

Gilmer J, Schoenholz SS, Riley PF, Vinyals O, Dahl GE (2017) Neural message passing for quantum chemistry. In: Proceedings of the 34th international conference on machine learning. Proceedings of machine learning research, vol 70. pp 1263–1272 https://proceedings.mlr.press/v70/gilmer17a.html

Grover A, Leskovec J (2016) node2vec: scalable feature learning for networks. In: Proceedings of the 22nd ACM SIGKDD international conference on knowledge discovery and data mining, pp. 855–864 ACM

Hong Y-l, Zhang Q-S (2016) Research on affinity propagation algorithm based on common neighbors. In: 2016 IEEE international conference on systems, man, and cybernetics (SMC), pp 1–6 https://doi.org/10.1109/SMC.2016.7844776

Hui Z, Xin L, Xiangju L, Zhongying Z, Chao L (2023) Adversarial variational autoencoder for attributed graph embedding with high-frequency noise filtering. Appl Intell 53:26750–26762. https://doi.org/10.1007/s10489-023-04961-2

Ikenna O, Hamida S, Mohammed H (2022) Improving node embedding by a compact neighborhood representation. Neural Comput Appl 35:7035–7048

Ikenna O, Hamida S, Mohammed H (2023) Identity2vec: learning mesoscopic structural identity representations via poisson probability metric. Int J Data Sci Anal 17:249–260

Isabelle K, Matthias T (2019) Using graph convolutional networks for approximate reasoning with abstract argumentation frameworks: a feasibility study. In: The 13th international conference on scalable uncertainty management (SUM'19), pp 24–37

Jie C, Tengfei M, Cao X (2018) Fastgcn: Fast learning with graph convolutional networks via importance sampling. arXiv:abs/1801.10247

Mikolov T, Chen K, Corrado GS, Dean J (2013) Efficient estimation of word representations in vector space. Computing research repository (CoRR) arXiv:abs/1301.3781

Oluigbo I, Haddad M, Seba H (2019) Evaluating network embedding models for machine learning tasks. In: Proceedings of international conference on complex networks and their applications VIII, SCI 881, Springer, pp 915–927

Perozzi B, Al-Rfou R, Skiena S (2014) Deepwalk: Online learning of social representations. In: Proceedings of the 20th ACM SIGKDD international conference on knowledge discovery and data mining, ACM, pp 701–710

Petar V, Guillem C, Arantxa C, Adriana R, Pietro L, Yoshua B (2017) Graph attention networks. arXiv:abs/1710.10903

Rebecca A, Simon B, Russell D, Frank W (2015) More reliable inference for the dissimilarity index of segregation. Economet J 18:40–66

Rossi RA, Ahmed NK (2015) The network data repository with interactive graph analytics and visualization. In: AAAI https://networkrepository.com

Sambaran B, Harsh K, Aswin K, Narasimha M (2018) Fscnmf: Fusing structure and content via non-negative matrix factorization for embedding information networks. arXiv:abs/1804.05313

Tenenbaum JB, Silva V, Langford JC (2000) A global geometric framework for nonlinear dimensionality reduction. Science 290:2319–2323

Thomas K, Max W (2016) Variational graph auto-encoders. arXiv:abs/1611.07308

Tu K, Cui P, Wang X, Wang F, Zhu W (2017) Structural deep embedding for hyper-networks. arXiv preprint arXiv:1711.10146

Wang D, Cui P, Zhu W (2016) Structural deep network embedding. In: Proceedings of the 22nd ACM SIGKDD international conference on knowledge discovery and data mining KDD '16, ACM, pp 1225–1234 https://doi.org/10.1145/2939672.2939753

Wei-Lin C, Xuanqing L, Si S, Yang L, Samy B, Cho-Jui H (2019) Cluster-gcn: an efficient algorithm for training deep and large graph convolutional networks. Proceedings of the 25th ACM SIGKDD international conference on knowledge discovery & data mining, pp 257–266

Yanning S, Traganitis PA, Giannakis GB (2018) Nonlinear dimensionality reduction on graphs. In: 2017 IEEE 7th international workshop on computational advances in multi-sensor adaptive processing, CAMSAP 2017, pp 1–5 https://doi.org/10.1109/CAMSAP.2017.8313127

Zhilin Y, William C, Ruslan S (2016) Revisiting semi-supervised learning with graph embeddings. In: Proceedings of the 33rd international conference on machine learning, vol 48. pp 1–9

Authors and Affiliations

Ikenna Oluigbo[1] · Stefan Duffner[1] · Kajal Eybpoosh[1] · Catherine Pothier[1]

✉ Stefan Duffner
stefan.duffner@insa-lyon.fr

Ikenna Oluigbo
ikenna-victor.oluigbo@insa-lyon.fr

Kajal Eybpoosh
kajal.eybpoosh@insa-lyon.fr

Catherine Pothier
catherine.pothier@insa-lyon.fr

[1] INSA Lyon, CNRS, LIRIS, UMR5205, 69622 Villeurbanne, France